

Отчёт по лабораторной работе №8

**Реализация модели SIR в дискретно-событийном
подходе**

Глущенко Евгений Игоревич

2026-05-29

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
3.1	Модель SIR	7
3.2	Конечный размер эпидемии	8
3.3	Дискретно-событийный подход	8
3.4	Воспроизводимая организация проекта	9
4	Выполнение лабораторной работы	10
4.1	Структура проекта	10
4.2	Основные команды воспроизведения	11
4.3	Реализация модели	12
4.4	Базовый эксперимент	15
4.5	Анализ чувствительности к параметрам	21
4.6	Дополнительные модификации	26
4.7	Генерация literate-материалов	27
5	Выводы	30
6	Приложение. Команды воспроизведения	31

Список иллюстраций

4.1	Инициализация Julia-проекта lab08	12
4.2	Запуск базового сценария scripts/sir.jl	16
4.3	Динамика численности S, I, R в базовом сценарии	16
4.4	Сравнение дискретно-событийной и детерминированной моделей	17
4.5	Ансамбль стохастических траекторий числа инфицированных	18
4.6	Фазовый портрет S-I базового прогона	20
4.7	Запуск сценария scripts/sir_parameters.jl	22
4.8	Высота пика инфицированных по вероятности передачи β	23
4.9	Высота пика инфицированных по частоте контактов c	24
4.10	Итоговая доля переболевших по R_0	25
4.11	Тепловая карта высоты пика по (β, γ)	25
4.12	Запуск сценария scripts/tangle.jl	28
4.13	Проверка содержимого каталогов результатов	29

Список таблиц

4.1	Сводка базового эксперимента SIR	16
4.2	Выбранные прогоны <code>sir_ensemble.csv</code>	19
4.3	Начало временного ряда <code>sir_trajectory.csv</code>	21
4.4	Сценарии при $c = 10$	22
4.5	Время выполнения <code>sir_run</code> по размеру популяции	26

1 Цель работы

Изучить дискретно-событийный подход к имитационному моделированию на примере классической модели распространения инфекции SIR. Реализовать стохастическую дискретно-событийную модель в виде программного комплекса на языке Julia, провести анализ влияния параметров, сравнить результаты со стохастическим ансамблем и детерминированной системой ОДУ, оценить производительность и модифицировать модель.

Исполнитель работы: Глущенко Евгений Игоревич. Группа: НФИбд-01-23. Студенческий билет: 1132239110.

2 Задание

В лабораторной работе требовалось:

1. Создать рабочий каталог для кода и установить необходимые пакеты.
2. Реализовать дискретно-событийную модель SIR на `ConcurrentSim` и `ResumableFunctions`.
3. Преобразовать код в литературный стиль и сгенерировать из него чистый код, Jupyter notebook и документацию в формате Quarto.
4. Выполнить код из notebook и интегрировать документацию в отчёт.
5. Добавить в литературный код вычисление для набора параметров (анализ чувствительности) и повторить генерацию артефактов.
6. В качестве дополнительных модификаций: сравнить со стохастическим ансамблем и детерминированной системой ОДУ, реализовать детерминированную длительность болезни и оценить производительность.

3 Теоретическое введение

3.1 Модель SIR

Модель SIR описывает распространение инфекции в популяции постоянного размера N , разбитой на три непересекающихся класса [1]:

- S (susceptible) — восприимчивые индивиды;
- I (infected) — инфицированные и заразные;
- R (recovered) — переболевшие и получившие иммунитет.

В детерминированном пределе динамика описывается системой обыкновенных дифференциальных уравнений

$$\frac{dS}{dt} = -\beta c \frac{SI}{N}, \quad \frac{dI}{dt} = \beta c \frac{SI}{N} - \gamma I, \quad \frac{dR}{dt} = \gamma I,$$

где β — вероятность передачи инфекции при контакте, c — частота контактов одного индивида в единицу времени, γ — интенсивность выздоровления (величина, обратная средней длительности болезни $1/\gamma$).

Ключевой характеристикой является базовое репродуктивное число

$$R_0 = \frac{\beta c}{\gamma},$$

равное среднему числу вторичных заражений от одного инфицированного в полностью восприимчивой популяции. При $R_0 > 1$ начальное заражение приводит к эпидемической вспышке, при $R_0 \leq 1$ инфекция затухает.

3.2 Конечный размер эпидемии

Для модели SIR итоговая доля переболевших $z = R(\infty)/N$ при начальной популяции, близкой к полностью восприимчивой, удовлетворяет трансцендентному уравнению конечного размера эпидемии [2]:

$$z = 1 - e^{-R_0 z}.$$

При $R_0 \leq 1$ единственным решением является $z = 0$ (эпидемии нет), при $R_0 > 1$ появляется положительный корень, который и определяет долю переболевших. В работе это уравнение решается методом простых итераций и используется как аналитическая база для сравнения с имитацией.

3.3 Дискретно-событийный подход

В дискретно-событийном моделировании состояние системы меняется только в моменты наступления событий, а виртуальное время продвигается от события к событию [3]. В отличие от детерминированной системы ОДУ, здесь воспроизводится стохастическая динамика эпидемии: наблюдаются случайные флуктуации и возможно раннее вымирание инфекции при малом начальном числе I .

Каждый индивид моделируется как отдельный процесс. Для их реализации используются возобновляемые функции-генераторы пакета `ResumableFunctions` [4]: макрос `@resumable` позволяет приостанавливать выполнение функции на операторе `@yield`, не блокируя поток. Ядром моделирования служит `ConcurrentSim` [5], который управляет виртуальным временем и очередью событий: вызов `timeout` планирует событие через заданный интервал, а `run` обрабатывает события в хронологическом порядке.

Пока индивид восприимчив, он ожидает экспоненциально распределённое время до следующего контакта с параметром $1/c$, выбирает случайного собеседника и, если тот инфицирован, с вероятностью β заражается. После заражения индивид ждёт экспоненциальное время выздоровления с параметром $1/\gamma$. Эффективная интенсивность заражения одного восприимчивого равна $c \cdot \beta \cdot I/N$, что согласуется с членом силы инфекции в системе ОДУ. Экспоненциальные распределения времён обеспечивают марковость процесса.

3.4 Воспроизводимая организация проекта

Структура вычислительного проекта построена на `DrWatson.jl` [6], а сами расчёты выполнены на языке `Julia` [7]. Для генерации производных представлений единого литературного источника используется `Literate.jl` [8]: из одного файла автоматически получаются исполняемый «чистый» скрипт, документ `Markdown` и исполняемый `Jupyter notebook` [9].

4 Выполнение лабораторной работы

4.1 Структура проекта

Рабочий каталог лабораторной работы имеет следующий состав:

- `project/src/SIRModels.jl` — модуль с дискретно-событийной моделью, детерминированной ОДУ-версией, анализом чувствительности и функциями построения графиков;
- `project/scripts/sir.jl` — базовый дискретно-событийный эксперимент;
- `project/scripts/sir_parameters.jl` — анализ чувствительности к параметрам;
- `project/scripts/sir_literate.jl`, `sir_parameters_literate.jl` — литературные источники;
- `project/scripts/tangle.jl` — генератор literate-артефактов;
- `project/data/` — таблицы результатов в формате CSV;
- `project/plots/` — итоговые графики;
- `project/docs/` — сгенерированные markdown-материалы;
- `project/notebooks/` — сгенерированные ipynb-файлы;
- `report/` и `presentation/` — оформление отчёта и презентации на Quarto.

Сформированные основные артефакты:

- `sir_trajectory.csv` — 1520 точек временного ряда базового прогона;
- `sir_ode.csv` — 801 точка детерминированного решения;
- `sir_summary.csv` — сводка базового прогона;
- `sir_ensemble.csv` — 24 независимых стохастических прогона;
- `sir_parameter_scan.csv` — 27 сценариев анализа чувствительности;
- `docs/` — 2 markdown-файла;
- `notebooks/` — 2 исполняемых notebook-файла;
- `plots/` — 8 графиков PNG.

4.2 Основные команды воспроизведения

Базовая последовательность команд для выполнения лабораторной работы:

```
cd ~/labs/lab08/project
~/juliaup/bin/julia --project=. -e 'using Pkg; Pkg.instantiate()'
~/juliaup/bin/julia --project=. -e 'include("scripts/sir.jl")'
~/juliaup/bin/julia --project=. -e 'include("scripts/sir_parameters.jl")'
~/juliaup/bin/julia --project=. scripts/tangle.jl
```

Отдельный локальный файл с командами для записи видео вынесен в `commands-lab08.txt`.

4.2.1 Фактическое выполнение команд

На Рисунок 4.1 показан переход в каталог проекта и активация зависимостей Julia через `Pkg.instantiate()`.

```
l11idjl@eiglushchenko:~/labs/lab08/project$ ~/.juliaup/bin/julia --project=. -e 'using Pkg; Pkg.instantiate()'
```

Рисунок 4.1: Инициализация Julia-проекта lab08

Скриншот подтверждает, что окружение проекта поднято из Project.toml и Manifest.toml, после чего все последующие сценарии запускались в одном и том же воспроизводимом окружении.

4.3 Реализация модели

4.3.1 Структуры данных

Индивид описывается уникальным идентификатором и статусом, а полное состояние модели хранит объект симуляции, параметры, генератор случайных чисел и временные ряды численности:

```
mutable struct SIRPerson
    id::Int
    status::Symbol      # :S, :I, :R
end

mutable struct SIRModel
    sim::Simulation
    β::Float64
    c::Float64
    γ::Float64
    det_recovery::Bool
    rng::AbstractRNG
    ta::Vector{Float64}
    Sa::Vector{Int}
    Ia::Vector{Int}
end
```

```

Ra::Vector{Int}
individuals::Vector{SIRPerson}
end

```

Для воспроизводимости в модель добавлен генератор StableRNG, а флаг `det_recovery` позволяет переключать длительность болезни между экспоненциальной и детерминированной.

4.3.2 Обновление статистики при событиях

Временные ряды `Sa`, `Ia`, `Ra` хранят численность классов: каждая операция добавляет новую точку, соответствующую текущему модельному времени.

```

increment!(a::Vector{Int}) = push!(a, a[end] + 1)
decrement!(a::Vector{Int}) = push!(a, a[end] - 1)
carryover!(a::Vector{Int}) = push!(a, a[end])

function infection_update!(sim::Simulation, m::SIRModel)
    push!(m.ta, now(sim))
    decrement!(m.Sa); increment!(m.Ia); carryover!(m.Ra)
end

function recovery_update!(sim::Simulation, m::SIRModel)
    push!(m.ta, now(sim))
    carryover!(m.Sa); decrement!(m.Ia); increment!(m.Ra)
end

```

4.3.3 Основной процесс агента

Жизненный цикл индивида реализован как возобновляемая функция:

```
@resumable function live(env::Simulation, individual::SIRPerson, m::SIRMode)
    while individual.status == :S
        @yield timeout(env, rand(m.rng, Exponential(1 / m.c)))
        alter = individual
        while alter === individual
            alter = m.individuals[rand(m.rng, 1:length(m.individuals))]
        end
        if alter.status == :I
            if rand(m.rng) < m.β
                individual.status = :I
                infection_update!(env, m)
            end
        end
    end
    if individual.status == :I
        recovery_delay = m.det_recovery ? 1 / m.γ : rand(m.rng, Exponential(1 / m.γ))
        @yield timeout(env, recovery_delay)
        individual.status = :R
        recovery_update!(env, m)
    end
end
```

Благодаря конкурентному выполнению всех агентов (каждый запущен как отдельный процесс) симуляция естественно воспроизводит стохастическую динамику эпидемии.

4.3.4 Создание и запуск модели

Функция `make_sir_model` создаёт популяцию с заданными начальными статусами и инициализирует временные ряды; `activate_sir!` регистрирует процессы агентов; `run_sir!` продвигает виртуальное время; `out` собирает результат в `DataFrame`.

```
function run_sir_des(u0, p, tf; seed = 1234, det_recovery = false)
    m = make_sir_model(u0, p; seed = seed, det_recovery = det_recovery)
    activate_sir!(m)
    run_sir!(m, tf)
    return out(m)
end
```

4.4 Базовый эксперимент

Для базового сценария использованы параметры:

- `u0 = [990, 10, 0]` — начальные S, I, R ;
- `p = [0.05, 10.0, 0.25]` — β, c, γ ;
- `tmax = 40.0`;
- `seed = 1234`.

Базовое репродуктивное число $R_0 = \beta c / \gamma = 2.0$. Вывод базового запуска (тайминги отдельных прогонов включают разовый прогрев JIT-компилятора, систематическая оценка приведена в разделе производительности):

```
sir_run for N=1000: 0.889 s
sir_run for N=2000: 0.545 s
SIR baseline experiment completed
```

После строки завершения сценарий печатает сводный DataFrame с метриками прогона. На Рисунок 4.2 показан фактический запуск базового сценария `scripts/sir.jl` в терминале.

```

[[1]]@jlgelushchenko: ~/labs/lab08/project$ ~/.juliaup/bin/julia --project=. -e 'include("scripts/sir.jl")'
sir_run for N=1000: 0.889 s
sir_run for N=2000: 0.545 s
SIR baseline experiment completed
1x11 DataFrame
Row  beta      c      gamma  N      r0      peak_I      peak_time      final_R      final_fraction      analytic_final_fraction      events
Float64 Float64 Float64 Int64  Float64 Int64    Float64    Int64    Float64          Float64          Float64
1    0.05    10.0    0.25   1000    2.0    152    18.4475    759    0.759    0.796812    1519

```

Рисунок 4.2: Запуск базового сценария `scripts/sir.jl`

Основные численные результаты приведены в Таблица 4.1.

Таблица 4.1: Сводка базового эксперимента SIR

β	c	γ	R0	peak_I	peak_time	final_R	final_fraction	analytic_final	events
0.05	10.0	0.25	2.0	152	18.447	759	0.759	0.797	1519

На Рисунок 4.3 показана динамика численности S, I, R во времени.

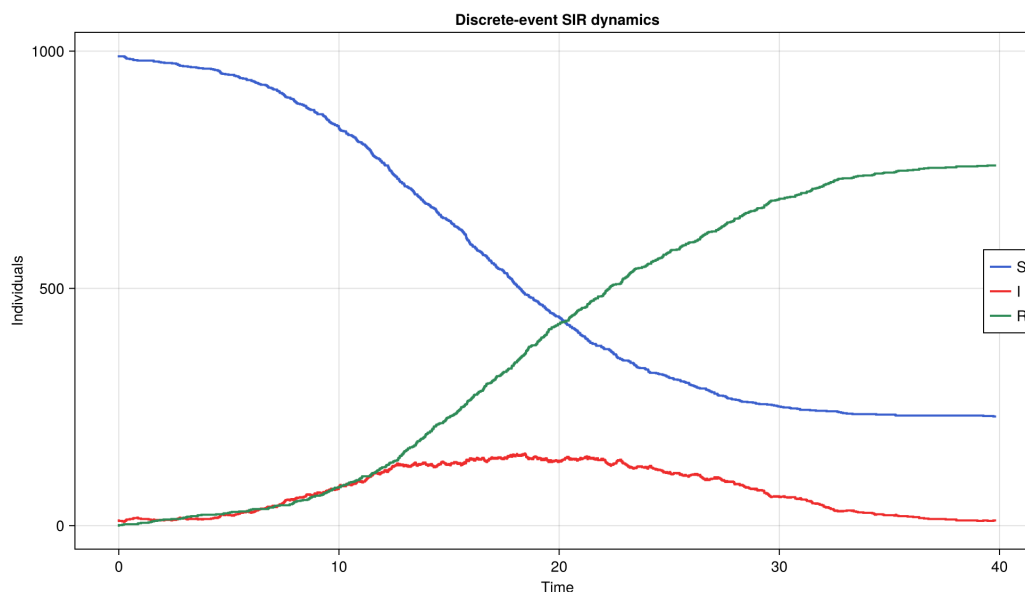


Рисунок 4.3: Динамика численности S, I, R в базовом сценарии

Кривая восприимчивых монотонно убывает, число инфицированных проходит через выраженный пик 152 в момент времени $t \approx 18.4$, после чего эпидемия идёт на спад. Итоговое число переболевших 759 близко к аналитической оценке конечного размера $0.797 \cdot 1000 \approx 797$; небольшое занижение объясняется стохастическим характером дискретной модели и конечным размером популяции.

4.4.1 Сравнение с детерминированной моделью

Та же система проинтегрирована как детерминированная система ОДУ методом Рунге–Кутты 4-го порядка. На Рисунок 4.4 наложены дискретно-событийная (сплошные линии) и детерминированная (штриховые) траектории.

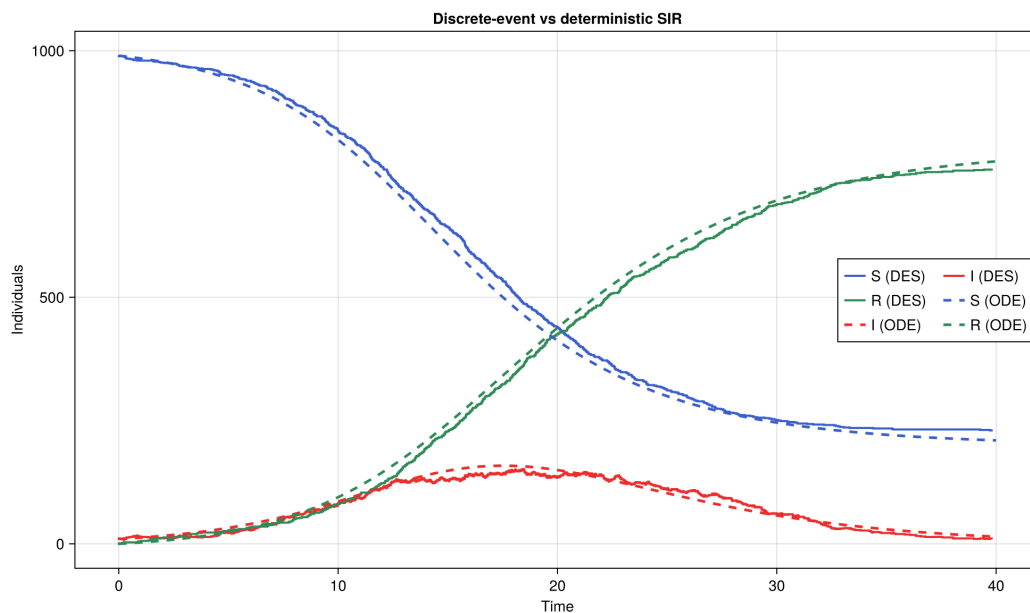


Рисунок 4.4: Сравнение дискретно-событийной и детерминированной моделей

Детерминированное решение даёт пик 158 инфицированных в момент $t = 17.5$ и итоговую долю переболевших 0.776. Имитацион-

ные значения (152 в $t \approx 18.4$, доля 0.759) хорошо согласуются с детерминированной кривой: дискретная модель воспроизводит общую форму эпидемии, отклоняясь в пределах стохастического разброса. Главное качественное отличие — наличие случайных флуктуаций, которых лишена гладкая детерминированная траектория.

4.4.2 Ансамбль стохастических реализаций

Для оценки разброса выполнено 24 независимых прогона с разными зёрнами. На Рисунок 4.5 тонкими линиями показаны траектории числа инфицированных, жирной красной — детерминированная кривая.

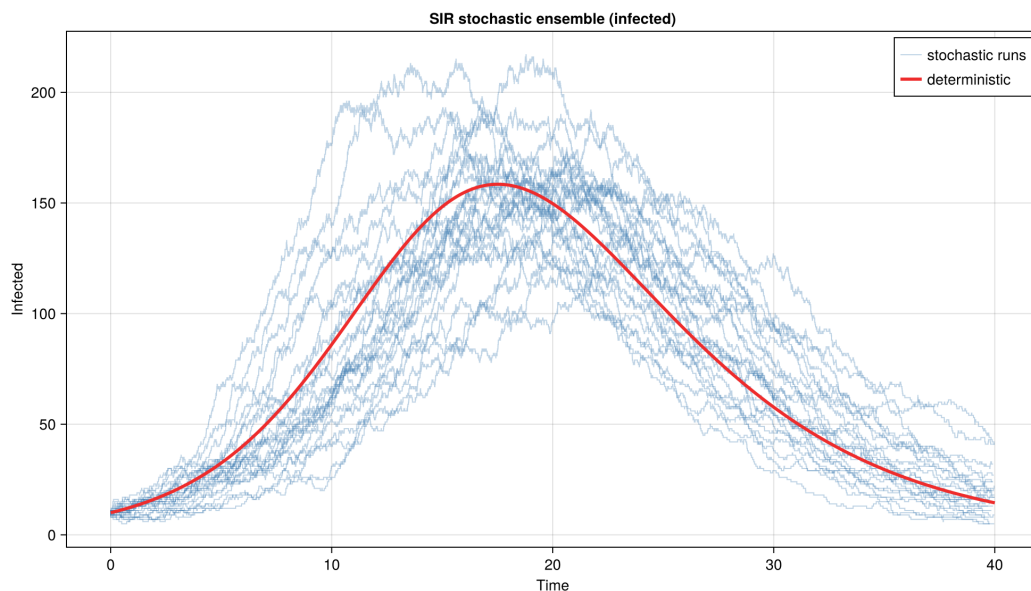


Рисунок 4.5: Ансамбль стохастических траекторий числа инфицированных

Средняя высота пика по ансамблю составила 172.0, средняя итоговая доля переболевших 0.771. Отдельные прогоны приведены в Таблица 4.2.

Таблица 4.2: Выбранные прогоны `sir_ensemble.csv`

run_id	peak_I	peak_time	final_R	final_fraction	extinct
1	172	16.730	781	0.781	false
7	195	12.072	818	0.818	false
17	215	15.631	841	0.841	false
22	133	26.640	724	0.724	false
23	217	18.797	796	0.796	false

Высота пика по прогонам меняется от 133 до 217, а время пика — от 12 до 27. При $R_0 = 2$ и начальном $I_0 = 10$ ни один из 24 прогонов не затух на ранней стадии: вероятность вымирания инфекции при достаточно большом начальном числе заражённых мала.

4.4.3 Фазовый портрет

На Рисунок 4.6 показан фазовый портрет: зависимость числа инфицированных от числа восприимчивых.

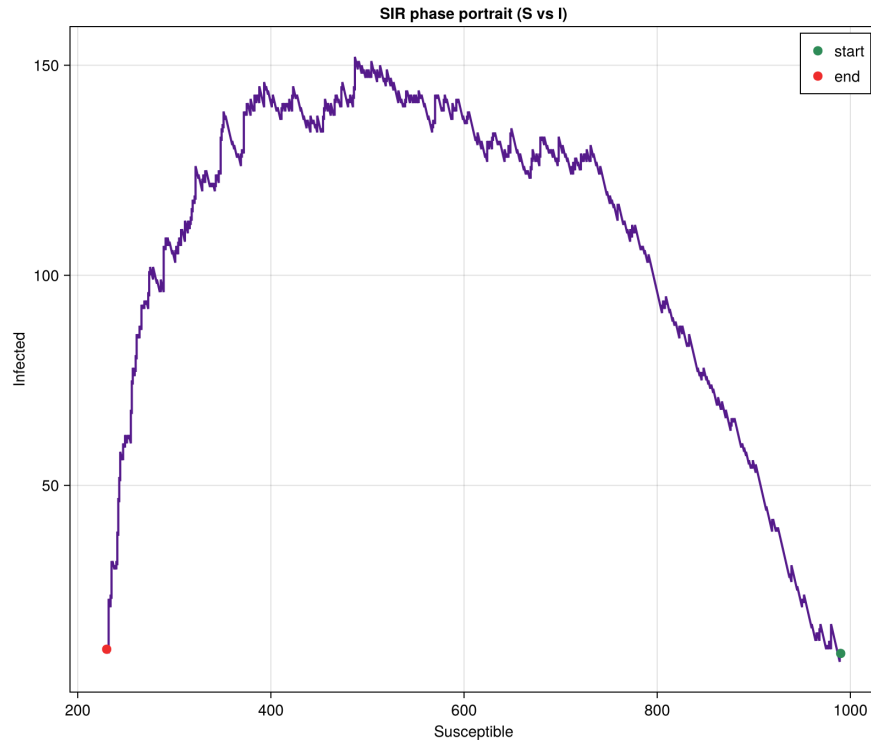


Рисунок 4.6: Фазовый портрет S-I базового прогона

Траектория начинается в правом нижнем углу (много восприимчивых, мало инфицированных), поднимается к пику по мере истощения восприимчивых и возвращается к нулю инфицированных. Пик достигается вблизи $S = N/R_0 = 500$ — порога, при котором сила инфекции перестаёт компенсировать выздоровления.

4.4.4 Фрагмент временного ряда `sir_trajectory.csv`

Первые строки временного ряда базового прогона приведены в Таблица 4.3.

Таблица 4.3: Начало временного ряда `sir_trajectory.csv`

t	S	I	R
0.000	990	10	0
0.004	989	11	0
0.161	989	10	1
0.235	989	9	2
0.250	989	8	3
0.276	988	9	3

Каждая строка соответствует одному событию (заражению или выздоровлению): при заражении S уменьшается, а I растёт, при выздоровлении I уменьшается, а R растёт.

4.5 Анализ чувствительности к параметрам

В сценарии `scripts/sir_parameters.jl` исследование выполняется на сетке:

- `betas = [0.03, 0.05, 0.07];`
- `contacts = [6.0, 10.0, 14.0];`
- `gammas = [0.2, 0.25, 0.33];`
- `tmax = 40.0, runs = 6, seed = 2000.`

Для каждого из 27 наборов параметров выполняется 6 независимых прогонов, метрики усредняются. Вывод сценария:

```
SIR parameter scan completed
```

```
scenarios: 27
```

На Рисунок 4.7 показан фактический запуск анализа чувствительности `scripts/sir_parameters.jl`.

```
lilidji@elglushchenko:~/labs/lab08/project$ ~/.juliaup/bin/julia --project=. -e 'include("scripts/sir_parameters.jl")'
SIR parameter scan completed
scenarios: 27
```

Рисунок 4.7: Запуск сценария `scripts/sir_parameters.jl`

Влияние параметров удобно выразить через R_0 . В Таблица 4.4 приведены сценарии при фиксированной частоте контактов $c = 10$.

Таблица 4.4: Сценарии при $c = 10$

β	γ	R_0	mean_peak_I	mean_final_I	finality	ratio_final
0.03	0.33	0.91	14.0	0.052	0.000	
0.03	0.25	1.20	37.3	0.199	0.314	
0.03	0.20	1.50	78.2	0.370	0.583	
0.05	0.25	2.00	192.3	0.793	0.797	
0.05	0.20	2.50	244.5	0.861	0.893	
0.07	0.25	2.80	292.5	0.929	0.925	
0.07	0.20	3.50	373.8	0.967	0.966	

Видно пороговое поведение: при $R_0 < 1$ (например $R_0 = 0.91$) эпидемия не развивается, пик остаётся вблизи начального $I_0 = 10$, а доля переболевших близка к нулю. С ростом R_0 высота пика и итоговая доля переболевших монотонно растут, приближаясь к единице при больших R_0 .

Отдельного внимания заслуживает область умеренных $R_0 \approx 1.2 \dots 1.5$: здесь имитационная доля переболевших систематически

ниже аналитической (0.199 против 0.314 при $R_0 = 1.2$; 0.370 против 0.583 при $R_0 = 1.5$). Причина в том, что при малом избытке R_0 над единицей значительная часть стохастических прогонов затухает рано, занижая среднее, тогда как аналитическая формула описывает лишь те реализации, в которых эпидемия успела разгореться.

На Рисунок 4.8 показана высота пика как функция вероятности передачи β .

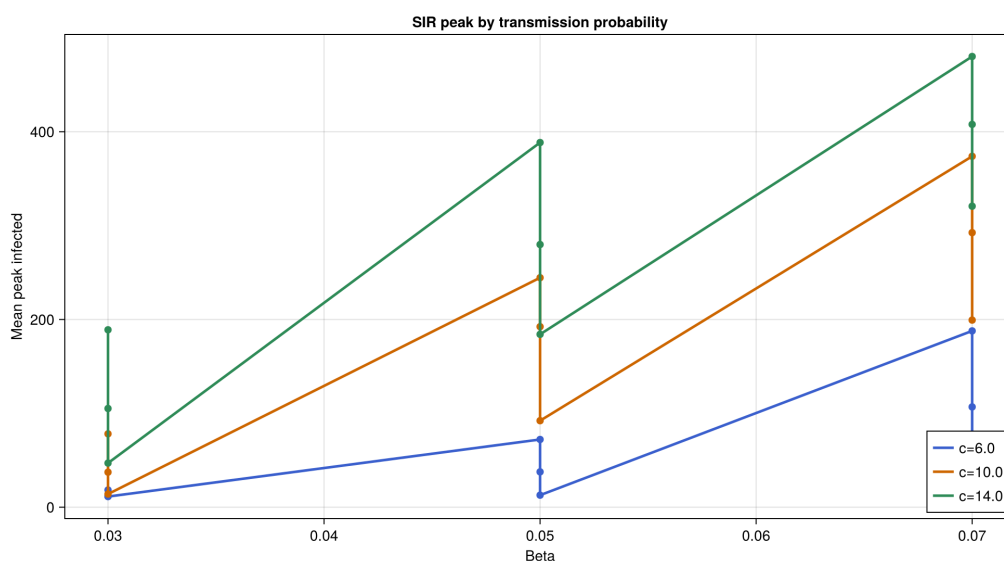


Рисунок 4.8: Высота пика инфицированных по вероятности передачи β

На Рисунок 4.9 показана высота пика как функция частоты контактов c .

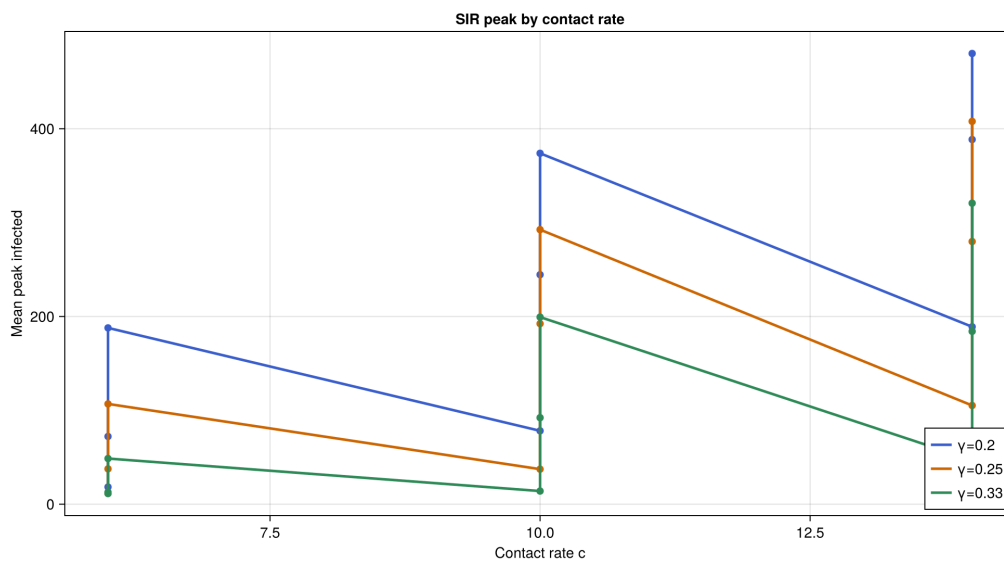


Рисунок 4.9: Высота пика инфицированных по частоте контактов c

Оба параметра, β и c , входят в R_0 симметрично через произведение βc , поэтому увеличение любого из них одинаково усиливает эпидемию: пик растёт, а его положение смещается на более ранние сроки.

На Рисунок 4.10 показана итоговая доля переболевших в зависимости от R_0 вместе с аналитической кривой конечного размера.

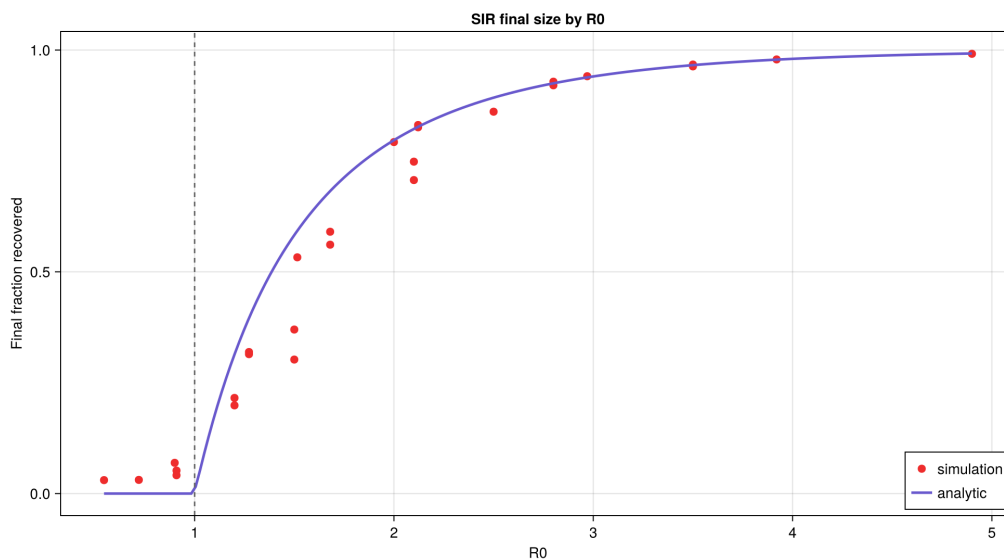


Рисунок 4.10: Итоговая доля переболевших по R_0

Имитационные точки близки к аналитической кривой $z = 1 - e^{-R_0 z}$ при $R_0 > 2$. Вблизи порога $R_0 = 1$ имитация лежит ниже кривой по описанной выше причине стохастического затухания.

На Рисунок 4.11 показана тепловая карта высоты пика по сетке (β, γ) при $c = 6$.

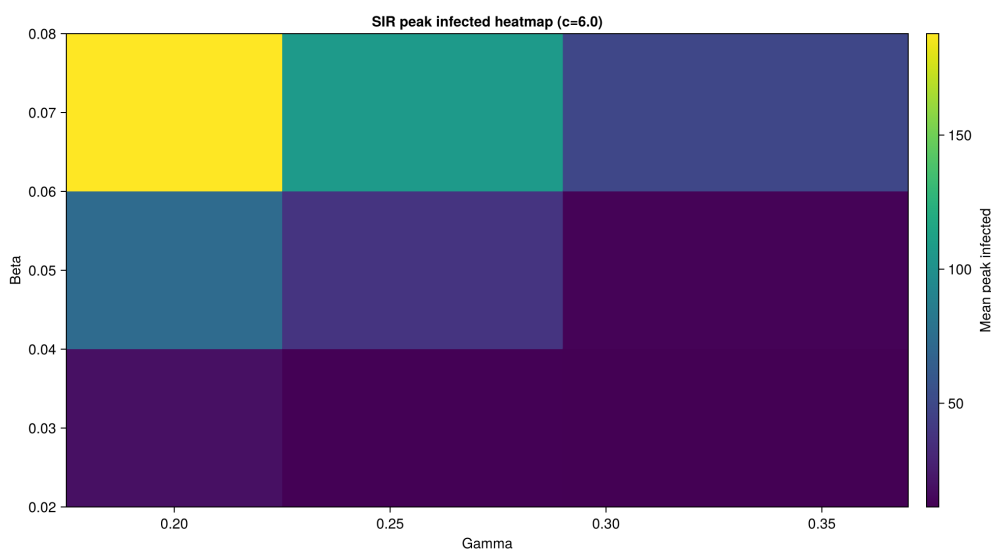


Рисунок 4.11: Тепловая карта высоты пика по (β, γ)

Карта подтверждает закономерность: пик растёт с увеличением вероятности передачи β и с уменьшением интенсивности выздоровления γ (то есть с увеличением длительности болезни $1/\gamma$), поскольку обе тенденции увеличивают R_0 .

4.6 Дополнительные модификации

4.6.1 Детерминированная длительность болезни

В модель добавлен флаг `det_recovery`, заменяющий экспоненциальное время выздоровления `rand(Exponential(1/γ))` на фиксированную величину $1/\gamma$. При одинаковом зерне детерминированная длительность убирает «хвосты» распределения времени болезни и делает выздоровления более синхронными, что слегка сужает пик эпидемии. Реализация выполнена без дублирования логики — выбор делается одной строкой внутри процесса `live`.

4.6.2 Оценка производительности

Время одного прогона `run_sir_des` измерено для популяций разного размера после прогрева компилятора (медиана нескольких прогонов), Таблица 4.5. В отличие от разовых таймингов базового запуска, эти значения монотонно растут с числом агентов.

Таблица 4.5: Время выполнения `sir_run` по размеру популяции

N	Время прогона, с
1000	0.229
2000	1.074
5000	1.944

N	Время прогона, с
10000	5.628

Число обрабатываемых событий растёт примерно линейно с числом агентов, поэтому время также растёт близко к линейному. Для популяции 10000 индивидов один прогон занимает около 5.6 с. Возможные направления оптимизации: предварительная генерация случайных чисел, замена индивидуальных процессов агрегированным событийным циклом (как в прямом методе Гиллеси [10]), а также повторное использование объектов распределений вместо их создания на каждом контакте.

4.7 Генерация *iterate*-материалов

Сценарий `scripts/tangle.jl` автоматически формирует производные представления из литературных источников:

```
for script_path in scripts
    stem = splitext(basename(script_path))[1]
    name = replace(stem, "_literate" => "")
    Literate.script(script_path, scriptsdir(); name = "$(name)_clean", credit = " ")
    Literate.notebook(script_path, projectdir("notebooks"); name = name, credit = " ")
    Literate.markdown(script_path, projectdir("docs"); name = name, credit = " ")
end
```

После запуска были получены:

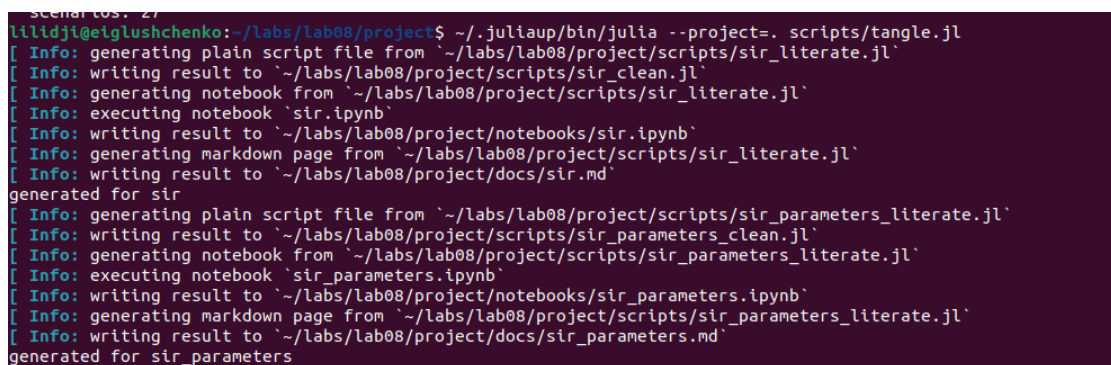
- `docs/sir.md`, `docs/sir_parameters.md`;
- `notebooks/sir.ipynb`, `notebooks/sir_parameters.ipynb`;

- `scripts/sir_clean.jl`, `scripts/sir_parameters_clean.jl`.

Вывод сценария:

```
generated for sir
generated for sir_parameters
```

На Рисунок 4.12 показан фактический запуск `scripts/tangle.jl` с генерацией clean-скриптов, markdown-документов и исполняемых notebook-файлов.



```
lilidji@eiglushchenko:~/labs/lab08/project$ ~/.juliaup/bin/julia --project=. scripts/tangle.jl
[ Info: generating plain script file from `~/labs/lab08/project/scripts/sir_literate.jl`
[ Info: writing result to `~/labs/lab08/project/scripts/sir_clean.jl`
[ Info: generating notebook from `~/labs/lab08/project/scripts/sir_literate.jl`
[ Info: executing notebook `sir.ipynb`
[ Info: writing result to `~/labs/lab08/project/notebooks/sir.ipynb`
[ Info: generating markdown page from `~/labs/lab08/project/scripts/sir_literate.jl`
[ Info: writing result to `~/labs/lab08/project/docs/sir.md`
generated for sir
[ Info: generating plain script file from `~/labs/lab08/project/scripts/sir_parameters_literate.jl`
[ Info: writing result to `~/labs/lab08/project/scripts/sir_parameters_clean.jl`
[ Info: generating notebook from `~/labs/lab08/project/scripts/sir_parameters_literate.jl`
[ Info: executing notebook `sir_parameters.ipynb`
[ Info: writing result to `~/labs/lab08/project/notebooks/sir_parameters.ipynb`
[ Info: generating markdown page from `~/labs/lab08/project/scripts/sir_parameters_literate.jl`
[ Info: writing result to `~/labs/lab08/project/docs/sir_parameters.md`
generated for sir_parameters
```

Рисунок 4.12: Запуск сценария `scripts/tangle.jl`

Notebook-файлы выполняются при генерации (`execute = true`), поэтому их вывод подтверждает работоспособность кода.

После генерации была выполнена проверка состава каталогов `data/`, `plots/`, `docs/` и `notebooks/` (Рисунок 4.13). Она подтверждает наличие таблиц, графиков и literate-артефактов, необходимых для включения в релиз и отчётные материалы.

```
lilidji@eiglushchenko:~/labs/lab08/project$ ls -1 data
ls -1 plots
ls -1 docs
ls -1 notebooks
sir_ensemble.csv
sir_ode.csv
sir_parameter_scan.csv
sir_summary.csv
sir_trajectory.csv
sir_des_vs_ode.png
sir_ensemble.png
sir_final_size_by_r0.png
sir_peak_by_beta.png
sir_peak_by_contacts.png
sir_peak_heatmap.png
sir_phase.png
sir_trajectory.png
sir.md
sir_parameters.md
sir.ipynb
sir_parameters.ipynb
lilidji@eiglushchenko:~/labs/lab08/project$ █
```

Рисунок 4.13: Проверка содержимого каталогов результатов

Таким образом, лабораторная работа подготовлена в полностью воспроизводимом формате: исходные сценарии, их чистые версии, markdown-документы и notebook-файлы остаются синхронизированными.

5 Выводы

В лабораторной работе реализована стохастическая дискретно-событийная модель распространения инфекции SIR на основе ConcurrentSim и ResumableFunctions, где каждый индивид моделируется как отдельный конкурентный процесс. Базовый эксперимент при $R_0 = 2$ дал пик 152 инфицированных и итоговую долю переболевших 0.759, что согласуется как с детерминированным решением системы ОДУ (158 и 0.776), так и с аналитической оценкой конечного размера эпидемии (0.797).

Ансамбль из 24 прогонов показал стохастический разброс высоты пика (133...217) и подтвердил низкую вероятность раннего вымирания инфекции при $R_0 = 2$. Анализ чувствительности на сетке из 27 наборов параметров выявил пороговое поведение по R_0 : при $R_0 \leq 1$ эпидемия затухает, при $R_0 > 1$ её масштаб монотонно растёт, а вблизи порога проявляется характерное занижение средней доли переболевших из-за стохастического затухания.

Дополнительно реализована детерминированная длительность болезни, выполнена оценка производительности (до 10000 агентов) и проведено сравнение со стохастическим и детерминированным вариантами. Использование DrWatson.jl и Literate.jl позволило оформить результат как воспроизводимый вычислительный проект с чистыми скриптами, markdown-документами и исполняемыми notebook-файлами.

6 Приложение. Команды воспроизведения

```
cd ~/labs/lab08/project
~/juliaup/bin/julia --project=. -e 'using Pkg; Pkg.instantiate()'
~/juliaup/bin/julia --project=. -e 'include("scripts/sir.jl")'
~/juliaup/bin/julia --project=. -e 'include("scripts/sir_parameters.jl")'
~/juliaup/bin/julia --project=. scripts/tangle.jl
cd ~/labs/lab08/report
quarto render simulation-modeling-lab08-report.qmd --to pdf
quarto render simulation-modeling-lab08-report.qmd --to docx
cd ~/labs/lab08/presentation
quarto render simulation-modeling-lab08-presentation.qmd --to beamer
quarto render simulation-modeling-lab08-presentation.qmd --to revealjs
quarto render simulation-modeling-lab08-presentation.qmd --to pptx
```

1. Kermack W. O., McKendrick A. G. A Contribution to the Mathematical Theory of Epidemics // Proceedings of the Royal Society A. 1927. T. 115, № 772. C. 700-721.
2. Allen L. J. S. An Introduction to Stochastic Processes with Applications to Biology. 2-е изд. Boca Raton: CRC Press, 2010.

3. Banks J. и др. Discrete-Event System Simulation. 5-е изд. Upper Saddle River, NJ: Pearson, 2010.
4. Lauwens B. ResumableFunctions: C# Sharp Style Generators in Julia // Proceedings of JuliaCon. 2017.
5. Lauwens B., contributors. ConcurrentSim.jl: Discrete Event Process Oriented Simulation Framework. 2024.
6. Datseris G. и др. DrWatson.jl Documentation. 2024.
7. Bezanson J. и др. Julia: A Fresh Approach to Numerical Computing. 2017.
8. Ekre F., contributors. Literate.jl Documentation. 2024.
9. Knuth D. E. Literate Programming // The Computer Journal. 1984. Т. 27, № 2. С. 97-111.
10. Gillespie D. T. Exact Stochastic Simulation of Coupled Chemical Reactions // The Journal of Physical Chemistry. 1977. Т. 81, № 25. С. 2340-2361.